

```
[28]: import pandas as pd
import numpy as np
from sklearn import model_selection
from sklearn import metrics
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
```

```
[29]: df = pd.read_csv("data/winequality-red_revised.csv")
```

```
[30]: df
```

```
[30]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.9	0.350	0.46	3.6	0.078	15.0	37	0.99730	3.35	0.86	12.80	8
1	10.3	0.320	0.45	6.4	0.073	5.0	13	0.99760	3.23	0.82	12.60	8
2	5.6	0.850	0.05	1.4	0.045	12.0	88	0.99240	3.56	0.82	12.90	8
3	12.6	0.310	0.72	2.2	0.072	6.0	29	0.99870	2.88	0.82	9.80	8
4	11.3	0.620	0.67	5.2	0.086	6.0	19	0.99880	3.22	0.69	13.40	8
...	...	...	...	...	...	...	...	...	...	...	...	...
295	7.6	1.580	0.00	2.1	0.137	5.0	9	0.99476	3.50	0.40	10.90	3
296	6.8	0.815	0.00	1.2	0.267	16.0	29	0.99471	3.32	0.51	9.80	3
297	7.3	0.980	0.05	2.1	0.061	20.0	49	0.99705	3.31	0.55	9.70	3
298	7.1	0.875	0.05	5.7	0.082	3.0	14	0.99808	3.40	0.52	10.20	3
299	6.7	0.760	0.02	1.8	0.078	6.0	12	0.99600	3.55	0.63	9.95	3

300 rows x 12 columns

```
[31]: # 결측치 확인
df.isnull().sum()
```

```
[31]: fixed acidity      0
volatile acidity      0
citric acid           0
residual sugar        0
chlorides             0
free sulfur dioxide    0
total sulfur dioxide   0
density              0
pH                  0
sulphates            0
alcohol              0
quality              0
dtype: int64
```

```
[32]: df['quality'].value_counts()
#csv파일을 변경하였고, quality가 고르고 분포되어있음.
```

```
[32]: quality
8    50
7    50
6    50
5    50
4    50
3    50
Name: count, dtype: int64
```

```
[33]: df['quality'].mean()
```

```
[33]: 5.5
```

```
[34]: df['quality'].max()
```

```
[34]: 8
```

```
[36]: df
```

[36]:	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.9	0.350	0.46	3.6	0.078	15.0	37	0.99730	3.35	0.86	12.80	2
1	10.3	0.320	0.45	6.4	0.073	5.0	13	0.99760	3.23	0.82	12.60	2
2	5.6	0.850	0.05	1.4	0.045	12.0	88	0.99240	3.56	0.82	12.90	2
3	12.6	0.310	0.72	2.2	0.072	6.0	29	0.99870	2.88	0.82	9.80	2
4	11.3	0.620	0.67	5.2	0.086	6.0	19	0.99880	3.22	0.69	13.40	2
...	...	...	...	...	...	...	...	...	...	...	...	...
295	7.6	1.580	0.00	2.1	0.137	5.0	9	0.99476	3.50	0.40	10.90	0
296	6.8	0.815	0.00	1.2	0.267	16.0	29	0.99471	3.32	0.51	9.80	0
297	7.3	0.980	0.05	2.1	0.061	20.0	49	0.99705	3.31	0.55	9.70	0
298	7.1	0.875	0.05	5.7	0.082	3.0	14	0.99808	3.40	0.52	10.20	0
299	6.7	0.760	0.02	1.8	0.078	6.0	12	0.99600	3.55	0.63	9.95	0

300 rows x 12 columns

```
[37]: # Input 변수(X)와 Output 변수(y) 구분
X = np.array(df.iloc[:, :-1])
y = np.array(df['quality'])
```

[38]: X

```
[38]: array([[ 7.9 ,  0.35 ,  0.46 , ...,  3.35 ,  0.86 , 12.8 ],
           [10.3 ,  0.32 ,  0.45 , ...,  3.23 ,  0.82 , 12.6 ],
           [ 5.6 ,  0.85 ,  0.05 , ...,  3.56 ,  0.82 , 12.9 ],
           ...,
           [ 7.3 ,  0.98 ,  0.05 , ...,  3.31 ,  0.55 ,  9.7 ],
           [ 7.1 ,  0.875,  0.05 , ...,  3.4 ,  0.52 , 10.2 ],
           [ 6.7 ,  0.76 ,  0.02 , ...,  3.55 ,  0.63 ,  9.95 ]])
```

```
[39]: y
```

```
[39]: array([2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
            2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
            2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
            2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
            2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
[40]: # 트레이닝 셋트와 테스트 셋트로 데이터 구분(여기서는 7:3으로 구분)
```

```
from sklearn.model_selection import train_test_split
```

random state는 반복적으로 같은 결과를 내기 위해서 설정

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
```

```
print("X_train 크기:", X_train.shape)
```

```
print("y train 크기:", y_train.shape)
```

```
print("X test 크기:", X_test.shape)
```

```
print("y_test 크기:", y_test.shape)
```

```
X_train 크기: (210, 11)
y_train 크기: (210,)
X_test 크기: (90, 11)
y_test 크기: (90,)
```

```
[41]: train_scores, test_scores = list(), list()
# train_score와 test_score의 정확도를 저장 장소
# train_score는 훈련데이터에 특화될수록 증가
# test_score는 테스트데이터에 일반화될수록 증가
```

```
[42]: # 의사결정나무모델에 데이터 적합(fitting)
from sklearn.tree import DecisionTreeClassifier

#의사결정나무 모델의 depth 조절(1부터 19까지)
for i in range(1,20):
    DT1 = DecisionTreeClassifier(max_depth=i)
    #깊이를 증가시키면서 의사결정나무 모델 적합
    DT1.fit(X_train, y_train)

    #training dataset 정확도 평가
    train_att = DT1.predict(X_train)
    train_acc = accuracy_score(y_train, train_att)
    train_scores.append(train_acc)

    #test dataset 정확도 평가
    test_att = DT1.predict(X_test)
    test_acc = accuracy_score(y_test, test_att)
    test_scores.append(test_acc)

    #train 정확도와 test 정확도를 depth 개수에 따른 정확도 확인
    print('>d, train: %.3f, test: %.3f' % (i, train_acc, test_acc))
```

```

>1, train: 0.576, test: 0.567
>2, train: 0.710, test: 0.633
>3, train: 0.767, test: 0.611
>4, train: 0.824, test: 0.633
>5, train: 0.890, test: 0.667
>6, train: 0.933, test: 0.722
>7, train: 0.962, test: 0.711
>8, train: 0.976, test: 0.700
>9, train: 0.986, test: 0.678
>10, train: 0.990, test: 0.678
>11, train: 0.995, test: 0.678
>12, train: 1.000, test: 0.667
>13, train: 1.000, test: 0.656
>14, train: 1.000, test: 0.656
>15, train: 1.000, test: 0.667
>16, train: 1.000, test: 0.656
>17, train: 1.000, test: 0.656
>18, train: 1.000, test: 0.667
>19, train: 1.000, test: 0.667

```

#그래프 그리기

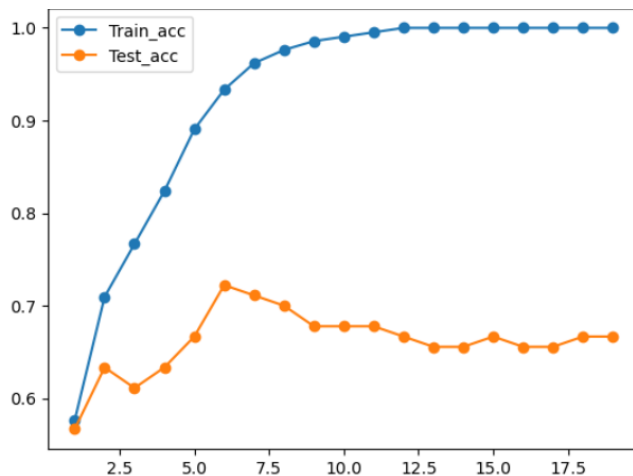
```
from matplotlib import pyplot
```

```

pyplot.plot(range(1,20), train_scores, '-o', label='Train_acc')
pyplot.plot(range(1,20), test_scores, '-o', label='Test_acc')
pyplot.legend()
pyplot.show()

```

#최적의 의사결정나무 깊이는? 6



```

[44]: # 의사결정나무모델에 데이터 적합(fitting)
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=0) #max_depth 0/지정
tree.fit(X_train, y_train)

```

```

[44]: DecisionTreeClassifier
DecisionTreeClassifier(random_state=0)

```

```
[45]: temp_y_pred = tree.predict(X_test)
#Training값을 X_test에서 불러옴
print('예측값\n', temp_y_pred)
print('실제값\n', y_test)
```

```
예측값
[1 1 2 0 1 1 0 0 2 2 1 2 1 0 0 2 0 2 0 1 1 1 1 2 0 1 1 0 0 0 2 2 2 1 2 2 1
 1 1 2 0 2 2 2 1 0 0 2 2 2 2 0 0 1 2 1 1 1 2 1 0 1 2 1 0 0 1 0 1 1 1 0 0 1
 0 2 2 2 0 1 1 1 1 0 2 1 2 0 0 1]
실제값
[0 1 2 0 0 1 0 0 2 1 0 2 1 0 2 2 0 2 0 1 1 1 1 1 0 1 1 0 0 0 2 2 2 1 0 1 1
 1 0 2 0 2 2 2 2 1 0 2 2 2 2 0 1 1 2 0 1 2 0 1 1 1 2 0 0 0 0 1 2 2 1 0 0 2
 1 0 2 2 1 1 1 1 0 0 0 2 0 0 0 0]
```

```
[46]: # 정확도 계산
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

temp_acc_before = accuracy_score(y_test, temp_y_pred)

print('정확도: ', format(temp_acc_before))

정확도: 0.6666666666666666
```

```
[47]: #의사결정나무의 깊이를 설정하여 재학습
#파라미터(max_depth)에 최적값이 갖 부여

tree = DecisionTreeClassifier(random_state=0, max_depth=6) #max_depth 6으로 지정
tree.fit(X_train, y_train)
```

```
[47]: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=6, random_state=0)
```

```
[48]: temp_y_pred = tree.predict(X_test)
#Training값을 X_test에서 불러옴
print('예측값\n', temp_y_pred)
print('실제값\n', y_test)
```

```
예측값
[0 1 2 0 1 1 0 2 2 2 1 2 1 0 0 2 0 2 0 1 1 1 1 2 0 1 1 0 0 0 2 2 2 1 0 2 1
 1 0 2 0 2 2 2 1 0 0 2 2 2 2 0 0 1 2 1 1 1 0 1 0 1 2 1 0 0 1 2 2 1 1 0 0 1
 0 2 2 2 0 1 1 1 1 2 0 1 2 0 0 0]
실제값
[0 1 2 0 0 1 0 0 2 1 0 2 1 0 2 2 0 2 0 1 1 1 1 1 0 1 1 0 0 0 2 2 2 1 0 1 1
 1 0 2 0 2 2 2 2 1 0 2 2 2 2 0 1 1 2 0 1 2 0 1 1 1 2 0 0 0 0 1 2 2 1 0 0 2
 1 0 2 2 1 1 1 1 0 0 0 2 0 0 0 0]
```

```
[49]: # 정확도 계산
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

temp_acc_after = accuracy_score(y_test, temp_y_pred)

print('정확도: ', format(temp_acc_after))

정확도: 0.7222222222222222
```

```
[54]: #최적화에 따른 정확도 비교

print('최적화 이전 정확도 : %.3f, 최적화 이후 정확도 : %.3f' % (temp_acc_before, temp_acc_after))

최적화 이전 정확도 : 0.667, 최적화 이후 정확도 : 0.722
```

```
[55]: #60211356 장유진
```

후기

지난주 와인 품질 검사 실습을 진행할 때에는 품질에 따른 데이터 전처리 과정에서 정확도가 바뀌는 것을 볼 수 있었다. 이번 주에는 최적화를 학습하고, 학습한 내용을 활용하여 다시 한번 와인 품질 검사 실습을 진행하였다.

최적화는 의사결정나무의 깊이를 조절하는 데에 사용된다. 실습 시에는 시각적 자료인 그래프를 통해 과적합에 가장 인접한 의사결정나무 깊이를 알 수 있도록 하였다.

와인 품질 검사 모델의 최적 의사결정나무 깊이는 6이었고, `max_depth`를 6으로 지정하여 정확도를 판단하였다.

그 결과, 최적화 작업을 거치지 않았을 때에는 정확도가 0.667이었지만, 최적화 작업을 거친 후의 정확도는 0.722로 향상되었다.

의사결정나무의 정확도를 높이는 것은 아주 중요하다. 그렇기에 정확도를 높이기 위한 다양한 방식을 알아두는 것이 좋겠다는 생각을 하였다.

지금까지 학습한 방식과 더불어 최적화는 유용하게 활용될 것이라고 생각한다.